

UNIVERSIDADE FEDERAL DO PIAUÍ

Campus Senador Helvídio Nunes de Barros

Bacharelado em Sistemas de Informação

Disciplina: Programação para Web II

Professor: Evandro J.R. Silva

Trabalho: APIs - GraphQL + WebSocket

Dupla: Rita de Cássia, Rogers Rocha Carvalho

1 RESUMO

Este trabalho apresenta a implementação de um sistema de monitoramento de recursos de computador (uso de CPU, RAM e disco), combinando dois paradigmas de API: WebSocket, para transmissão dos dados em tempo real, e GraphQL, para consultas flexíveis sobre o histórico recente das métricas coletadas. O servidor foi desenvolvido em Python com FastAPI e persiste os dados em um banco SQLite local; o cliente é uma página web simples que exibe os valores atualizados e permite disparar consultas GraphQL sob demanda. Além da implementação, este relatório documenta as duas APIs desenvolvidas e apresenta um estudo comparativo entre os paradigmas, considerando modelo de comunicação (push x pull), tamanho de payload e uso de recursos do servidor.

2 INTRODUÇÃO

APIs são o principal mecanismo pelo qual sistemas trocam dados entre si, e a escolha do paradigma de comunicação afeta diretamente características como latência, uso de banda e complexidade do cliente. Dados de monitoramento - como uso de CPU, RAM e disco - são um bom caso de uso para comparar paradigmas diferentes, pois exigem tanto atualização contínua (para acompanhar o estado atual da máquina) quanto consultas pontuais sobre o que já foi coletado.

O sistema foi desenvolvido utilizando a linguagem Python no back-end, com o framework FastAPI. Para a implementação da API GraphQL foi utilizada a biblioteca Strawberry GraphQL, enquanto a comunicação em tempo real foi realizada por meio de WebSockets. Os dados coletados são armazenados em um banco de dados SQLite. O sistema monitora três recursos principais do computador: CPU, memória RAM e armazenamento em disco. As informações são coletadas automaticamente pelo programa e armazenadas no banco de dados.

O objetivo deste trabalho é implementar uma API de cada uma das duas categorias exigidas: uma de transmissão (WebSocket) e uma de consulta (GraphQL); e, a partir dessa implementação concreta, comparar na prática os dois paradigmas. O restante do relatório documenta as tecnologias, a arquitetura e as especificações das APIs desenvolvidas.

3 TECNOLOGIAS UTILIZADAS

Foram utilizadas as seguintes tecnologias no desenvolvimento do projeto:

Tabela 1: Tecnologias utilizadas no projeto

Tecnologia	Função no projeto
Python	Linguagem utilizada no desenvolvimento do back-end.
FastAPI	Framework utilizado para disponibilizar o servidor web e os endpoints da aplicação.
Strawberry GraphQL	Biblioteca utilizada para implementação da API GraphQL.
WebSocket	Tecnologia utilizada para comunicação contínua e atualização dos dados.
SQLite	Banco de dados utilizado para armazenar as métricas coletadas.
psutil	Biblioteca utilizada para obter informações sobre CPU, RAM e disco.
HTML	Estrutura da interface web.
CSS	Estilização da interface.
JavaScript	Comunicação do navegador com GraphQL e WebSocket, e atualização da interface.

4 PARADIGMAS DE API ESCOLHIDOS

4.1 GraphQL

O primeiro paradigma escolhido foi o GraphQL (paradigma de consulta). Nele, o cliente especifica exatamente quais dados deseja receber, evitando *over-fetching* ou *under-fetching*. Neste projeto, o GraphQL foi utilizado principalmente para consultar informações armazenadas no banco de dados.

A API possui uma consulta para obter o histórico completo das métricas:

```
query {  
  metricas {  
    id  
    horario  
    cpu  
    ram  
    disco  
  }  
}
```

Também foi implementada uma consulta para calcular a média de utilização da CPU em um determinado período (ex: últimos 30 segundos):

```
query {  
  mediaCpu(segundos: 30)  
}
```

4.2 WebSocket

O segundo paradigma escolhido foi o WebSocket (paradigma de transmissão). Ele permite estabelecer uma conexão persistente entre cliente e servidor, ideal para o envio de dados em tempo real sem a necessidade de novas requisições HTTP a cada atualização.

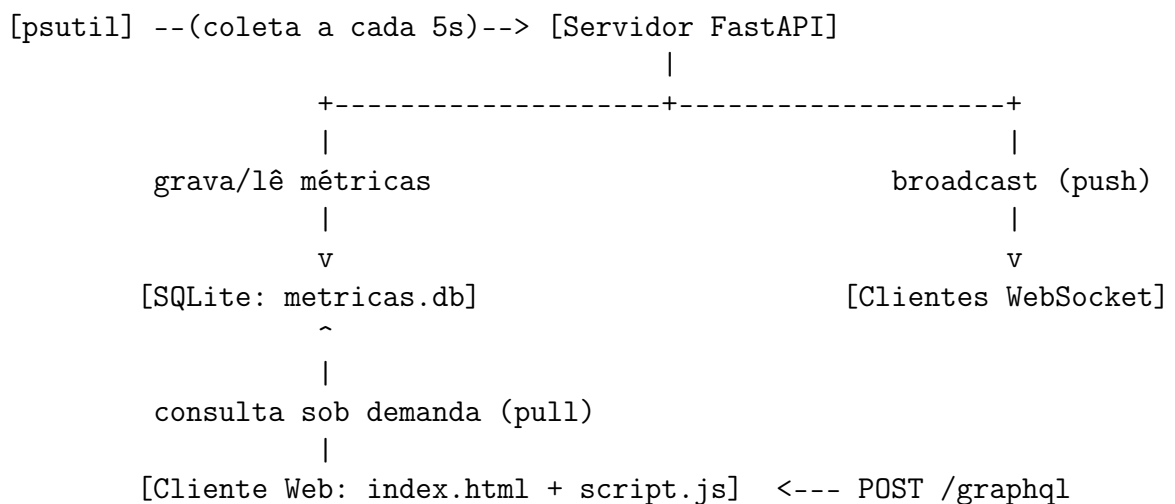
A conexão é estabelecida pelo navegador através do endpoint `/ws`. Quando o cliente acessa a página, o JavaScript cria a conexão da seguinte forma:

```
const websocket = new WebSocket(
    `${protocolo}://${window.location.host}/ws`
);
```

O servidor mantém uma lista dos clientes conectados e envia as novas métricas para eles continuamente.

5 ARQUITETURA DA APLICAÇÃO

Visão geral do fluxo de dados:



Estrutura de Diretórios:

A aplicação foi organizada em duas partes principais (back-end e front-end), estruturadas da seguinte forma:

```
GraphQLWebSocket/
|
+-- back/
|   +-- banco.py
|   +-- coletor.py
|   +-- graphql_api.py
|   +-- main.py
|   +-- websocket.py
|   +-- metricas.db
|   +-- teste.py
|   +-- requirements.txt
|
```

```
+-- front/
+-- index.html
+-- script.js
+-- style.css
```

Ciclo de coleta e retenção: a cada 5 segundos, o servidor lê CPU/RAM/disco, grava uma linha no banco e faz o envio aos clientes via WebSocket. Registros com mais de 5 minutos são descartados do banco automaticamente, mantendo o sistema rápido e funcionando estritamente como um *dashboard* em tempo real.

6 DOCUMENTAÇÃO DA API WEBSOCKET

Endpoint: `ws://127.0.0.1:8000/ws` (`wss://` se o site for https)

Fluxo e Formato da Mensagem (JSON):

1. O cliente abre a conexão. O servidor aceita e guarda em uma lista de clientes ativos.
2. A cada 5 segundos, o servidor envia (broadcast) o payload fixo abaixo para todos:

```
{
  "horario": "2026-09-14 10:15:30",
  "cpu": 32.5,
  "ram": 61.2,
  "disco": 45.1
}
```

7 DOCUMENTAÇÃO DA API GRAPHQL

Endpoint: `POST /graphql` (Content-Type: application/json)

Schema (Query):

- `metricas: [Metrica]`
Retorna todo o histórico armazenado (até 5 min).
- `mediaCpu(segundos: Int!): Float`
Média de uso de CPU nos últimos N segundos.
- `mediaRam(segundos: Int!): Float`
Média de uso de RAM nos últimos N segundos.
- `mediaDisco(segundos: Int!): Float`
Média de uso de disco nos últimos N segundos.
- `estatisticasCpu(segundos: Int!): EstatisticasCpu`
`EstatisticasCpu { media, maximo, minimo, amostras }`

Exemplo de requisição combinada:

```
query {  
  estatisticasCpu(segundos: 60) {  
    media  
    maximo  
    minimo  
  }  
}
```

8 Telas da Aplicação

Esta seção apresenta as capturas de tela demonstrando o funcionamento integrado do sistema de monitoramento, incluindo a interface do cliente e o log de execução do servidor.

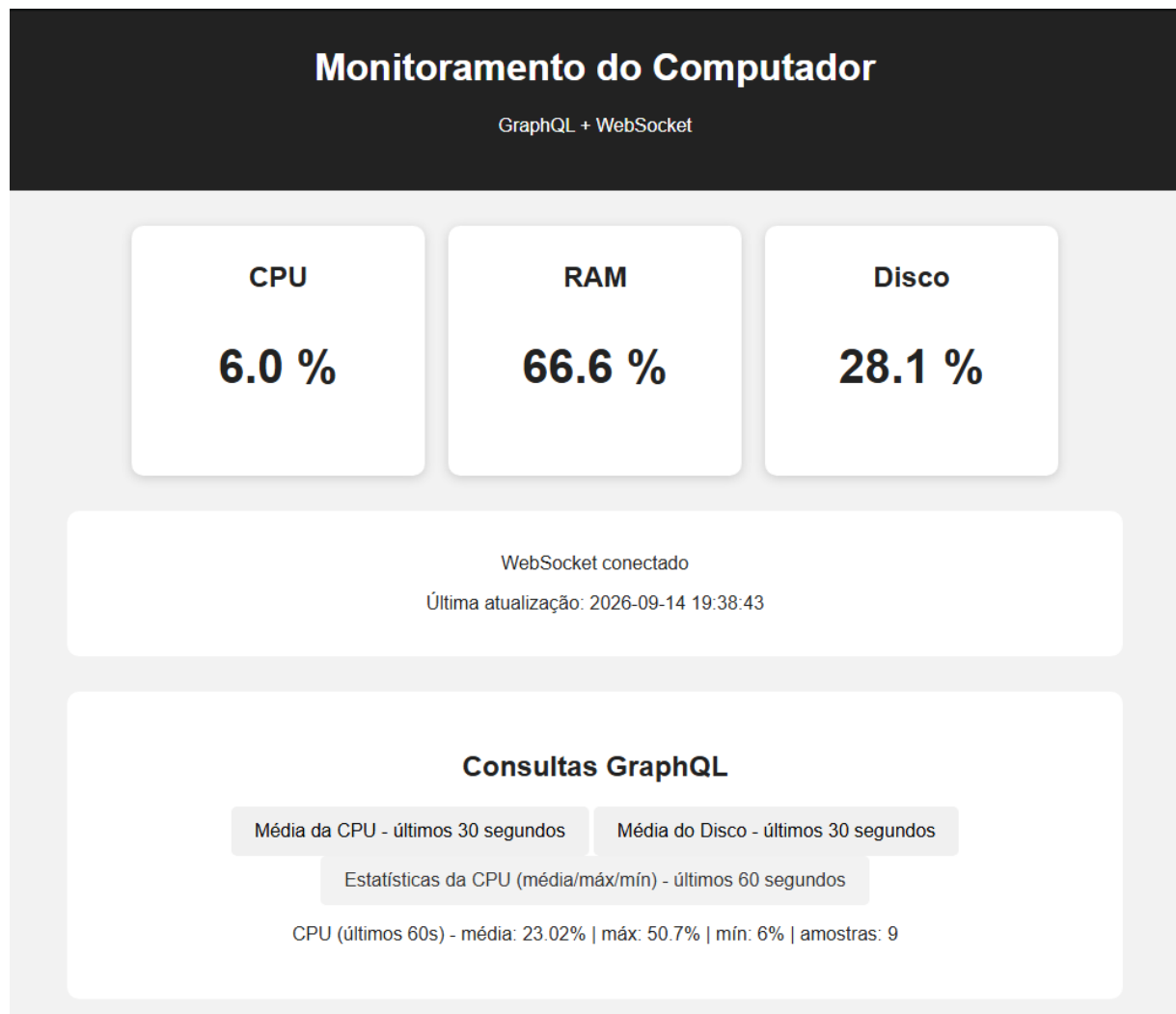


Figura 1: Dashboard principal de monitoramento atualizado em tempo real via WebSocket.



Figura 2: Interface do cliente exibindo a estrutura para resultados de consultas GraphQL.

```
Windows PowerShell - (.venv) back

(.venv) PS C:\Users\Rita\Downloads\GraphQLWebSocket-main\GraphQLWebSocket-main\back> py -m uvicorn main:app --
reload
INFO:      Will watch for changes in these directories: ['C:\\Users\\Rita\\Downloads\\GraphQLWebSocket-
main\\GraphQLWebSocket-main\\back']
INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:      Started reloader process [11724] using WatchFiles
INFO:      Started server process [11724]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
INFO:      127.0.0.1:52965 - "GET / HTTP/1.1" 200 OK
INFO:      127.0.0.1:53880 - "GET / HTTP/1.1" 200 OK
INFO:      127.0.0.1:53880 - "GET /static/style.css HTTP/1.1" 200 OK
INFO:      127.0.0.1:53329 - "GET /static/script.js HTTP/1.1" 200 OK
INFO:      127.0.0.1:49992 - "WebSocket /ws" [accepted]
INFO:      connection open
INFO:      127.0.0.1:56188 - "POST /graphql HTTP/1.1" 200 OK
```

Figura 3: Terminal do servidor FastAPI evidenciando a inicialização pelo Uvicorn e o tráfego de requisições.

9 ESTUDO COMPARATIVO: GRAPHQL x WEBSOCKET

9.1 Comparação conceitual

Aspecto	GraphQL	WebSocket
Modelo	Pull (cliente pede)	Push (servidor envia)
Uso ideal	Consultas históricas, agregadas, sob demanda	Dados em tempo real, streaming contínuo
Payload	Só os campos pedidos (evita over-fetching)	Objeto fixo, sempre o mesmo formato
Conexão	1 requisição HTTP por consulta	1 conexão persistente para toda a sessão
Overhead	Alto (handshake HTTP a cada chamada)	Baixo (só o frame com os dados)

9.2 Comparação prática

Os testes foram realizados em dois notebooks (i5-1135G7 e i5-13420H, ambos com 16 GB de RAM), executando cada API por cerca de um minuto e medindo uso de CPU/RAM do processo servidor, tamanho médio de payload e tempo de resposta.

O consumo de CPU e RAM do servidor foi próximo entre os dois paradigmas em ambas as máquinas, sem indicar que um deles é sistematicamente mais leve. Já o payload variou bastante: o WebSocket manteve mensagens pequenas e fixas (68 bytes), enquanto o payload do GraphQL (consulta `metrics`) dependeu de quantos registros já haviam se acumulado na janela de retenção de 5 minutos (Seção 5) no momento do teste – 2.234

bytes no Notebook 1, contra 629,64 bytes no Notebook 2, testado logo após o servidor subir.

Quanto ao tempo de resposta, o GraphQL teve latência média de requisição de 23,32 ms (Notebook 1) e 19,45 ms (Notebook 2), compatível com uma requisição HTTP simples. O WebSocket, por não ser requisição-resposta, não tem um equivalente direto: o valor medido (5.511,04 ms / 5.841,55 ms) é apenas o intervalo entre mensagens sucessivas, definido pelo ciclo de coleta de 5s do servidor – não uma latência de rede, e não deve ser comparado diretamente ao número do GraphQL.

Em suma, GraphQL e WebSocket tiveram uso de recursos semelhante, mas comportamentos de payload e tempo de resposta distintos, coerentes com seus modelos pull e push. Os resultados estão na tabela abaixo.

Métrica	Notebook 1 (GraphQL / WS)	Notebook 2 (GraphQL / WS)
CPU média (servidor)	0,13% / 0,08%	1,52% / 0,29%
RAM média (servidor)	33,46 MB / 33,43 MB	59,48 MB / 59,85 MB
Tamanho do payload	2.234 bytes / 68 bytes	629,64 bytes / 67,82 bytes
Latência média da requisição (GraphQL)	23,32 ms	19,45 ms
Intervalo médio entre mensagens (WebSocket)*	5.511,04 ms	5.841,55 ms
Especificação	i5-1135G7 / 16 GB	i5-13420H / 16 GB

*Métrica não comparável diretamente à latência do GraphQL: reflete o ciclo de coleta de 5 segundos configurado no servidor (Seção 4), não o tempo de entrega de uma mensagem individual.

9.3 Considerações finais

Os testes confirmaram que cada paradigma atende melhor a uma necessidade diferente: o WebSocket é mais adequado à transmissão em tempo real, por manter uma conexão persistente e permitir que o servidor envie automaticamente as novas métricas a cada coleta; já o GraphQL é mais adequado às consultas históricas, pois permite ao cliente pedir exatamente os dados de interesse (como médias de CPU/RAM em um período), reduzindo o payload quando a pergunta é pontual (Seção 9.2).

As principais dificuldades de implementação foram a integração entre coletor, banco SQLite, servidor FastAPI, schema GraphQL, conexões WebSocket e interface web – em especial, tratar corretamente clientes que se desconectam e garantir que cada métrica fosse persistida e transmitida de forma consistente. Outro ponto relevante foi o controle de armazenamento: como a coleta é contínua, o banco cresceria indefinidamente sem uma rotina que descarta automaticamente registros com mais de cinco minutos (Seção 5) – escolha adequada a um monitoramento "ao vivo", mas que limita o sistema a análises de curto prazo; um trabalho futuro poderia agregar dados antigos (ex.: médias por minuto) em vez de descartá-los.

Conclui-se que GraphQL e WebSocket são complementares, não concorrentes: juntos, atenderam às diferentes necessidades do sistema de monitoramento e permitiram comparar, na prática e com dados mensuráveis, as características de cada paradigma de API.

Código-fonte: <https://github.com/Rogers-Rocha/GraphQLWebSocket>